PRODUCT REQUIREMENTS DOCUMENT

Dance Studio Management Platform

CRM | Tuition Management | Scheduling | Parent Portal | Financial Planning

Version 1.0 — MVP
March 2026

Table of Contents

1. Executive Summary

This document defines the product requirements for a Dance Studio Management Platform — a comprehensive, web-based SaaS application designed to serve dance studios, academies, and independent dance companies. The platform consolidates student lifecycle management, class scheduling, tuition billing, parent portals, teacher tooling, financial planning, and public-facing web features into a single cohesive system.

The MVP (Phase 1) will deliver a fully functional, multi-role application covering all core workflows identified below, along with a public-facing website and embeddable schedule widget. The platform is designed to be AI-buildable inside a modern IDE (e.g., Cursor) from this document alone.

1.1 Product Vision

To become the single operating system for dance studios — eliminating the need for spreadsheets, disparate billing tools, and paper-based attendance by providing an elegant, integrated platform for owners, teachers, parents, and students.

1.2 Target Users

Role	Description	Access Level
Admin / Owner	Studio owner or business manager	Full access to all features
Teacher / Instructor	Dance instructors managing classes	Limited: own classes, attendance, student view
Parent / Guardian	Parents of enrolled students	Registration, payments, schedule, child history
Student (Adult)	Adult students managing their own account	Registration, payments, own schedule & history
Public / Visitor	Website visitors, prospective families	Public schedule, registration interest, pricing

2. Recommended Tech Stack

The following tech stack is chosen for developer ergonomics, type safety, rapid MVP delivery, production scalability, and compatibility with AI-assisted development in Cursor.

2.1 Frontend

Layer	Technology	Rationale
Framework	Next.js 14 (App Router)	SSR/SSG for public pages, React for app UI, file-based routing
Language	TypeScript	Type safety, better IDE autocompletion, fewer runtime bugs
Styling	Tailwind CSS + shadcn/ui	Utility-first CSS, accessible component primitives
Animation	Motion (motion.dev)	Production-grade React animation library; page transitions, list animations, modal/drawer enters, dashboard stat reveals. Must be used in 'use client' components only.
State Management	Zustand + React Query (TanStack)	Lightweight global state + server-state caching
Forms	React Hook Form + Zod	Type-safe forms with schema validation
Charts / Analytics	Recharts or Chart.js	Enrollment stats, revenue charts, utilization graphs
Calendar / Scheduling	FullCalendar (React)	Class scheduling display with drag/drop support
Payments UI	Stripe Elements	PCI-compliant hosted payment UI components
Embeddable Widget	Vanilla JS build via Next.js custom output	iframe-embeddable schedule + signup widget

2.1.1 Motion Animation Guidelines

Motion (motion.dev) is the designated animation library for all UI transitions and micro-interactions. Use it consistently across the app for a polished, cohesive feel.

- Install: npm install motion
- Import: import { motion, AnimatePresence } from 'motion/react'
- All motion components must live in files marked 'use client'
- Wrap animated list items in motion.div with layout prop for smooth reordering
- Use AnimatePresence for any element that conditionally mounts/unmounts (modals, drawers, toasts, empty states)
- Page transitions: wrap (app) layout children in AnimatePresence with a shared fade/slide variant
- Dashboard stat cards: staggered fade-up on mount using variants and staggerChildren
- Do NOT animate inside Server Components — create a thin 'use client' wrapper around the animated part
- Respect prefers-reduced-motion: use the useReducedMotion() hook and skip or minimize animations when true

Recommended animation presets — define once in lib/motion.ts and reuse across the app:

- fadeUp — opacity 0->1, y 16->0, ease: easeOut, duration: 0.3

- fadeIn — opacity 0->1, duration: 0.2
- scaleIn — opacity 0->1, scale 0.95->1, duration: 0.2 (modals and popovers)
- staggerContainer — staggerChildren: 0.07, delayChildren: 0.1
- slideInRight — x 24->0, opacity 0->1 (drawers and side panels)

2.2 Backend

Layer	Technology	Rationale
API Framework	Next.js API Routes / Route Handlers	Collocated with frontend, serverless-compatible
ORM	Prisma	Type-safe DB client, schema migrations, excellent DX
Database	PostgreSQL (via Supabase or Neon)	Relational data model ideal for CRM & billing
Auth	NextAuth.js (Auth.js v5)	Multi-role auth, OAuth + email/password, session mgmt
File Storage	Supabase Storage or AWS S3	Profile photos, documents, receipts
Email	Resend + React Email	Transactional emails: invoices, receipts, reminders
SMS (Optional)	Twilio	Class reminders, payment due alerts
Background Jobs	Inngest or Vercel Cron	Recurring billing, reminder emails, report generation
PDF Generation	React PDF (@react-pdf/renderer)	Invoice and receipt PDF generation

2.3 Payments

Concern	Solution
Payment Processing	Stripe (Checkout, Payment Intents, Customer Portal)
Recurring Tuition	Stripe Subscriptions or scheduled Payment Intents
Invoicing	Stripe Invoices API + custom PDF overlay
Family Discounts	Stripe Coupon / Discount objects applied at checkout
Refunds	Stripe Refund API
Scholarships	Internal credit ledger applied pre-Stripe charge

2.4 Infrastructure & DevOps

Concern	Solution
Hosting	Vercel (frontend + API routes) — zero-config Next.js deployment
Database Hosting	Supabase (Postgres + Auth + Storage) or Neon (serverless Postgres)
CI/CD	GitHub Actions — lint, type check, test, deploy on merge to main

Concern	Solution
Environment Config	dotenv + Vercel Environment Variables
Error Monitoring	Sentry
Analytics	PostHog (self-hostable, privacy-first)
Version Control	Git + GitHub

3. Data Model & Database Schema

The following describes the core Prisma schema entities and their relationships. This is the source of truth for the data model the development team should implement.

3.1 Core Entities Overview

Entity	Description
User	All platform users (admin, teacher, parent, student). Auth identity.
StudioProfile	The studio/organization configuration and branding
Student	A person enrolled in classes. Can be linked to a User (adult) or a Guardian.
Guardian	Parent or guardian with one or more Student dependents
FamilyGroup	Groups related Students and Guardians for discount and billing purposes
Teacher	Instructor profile linked to a User account
Room	Physical studio rooms or spaces with name, capacity, description
ClassTemplate	Reusable class definition: name, description, style, level, pricing
ClassSession	A scheduled instance of a ClassTemplate with room, teacher, time
Enrollment	M:M junction between Student and ClassSession
Attendance	Per-session attendance record per enrolled Student
Invoice	Billing record for tuition or custom charges
InvoiceLineItem	Individual line items on an invoice
Payment	Records of payments made against invoices (via Stripe)
Scholarship	Scholarship or credit awarded to a Student or FamilyGroup
PricingRule	Custom pricing logic including family discounts
AuditLog	Immutable log of all significant system actions

3.2 Key Relationships

- User 1:1 Student (adult self-enrolling)
- User 1:1 Guardian (parent portal user)
- Guardian 1:N Student (parent has multiple children)
- FamilyGroup 1:N Student (billing family unit)
- FamilyGroup 1:N Guardian
- Student N:M ClassSession via Enrollment
- ClassSession N:1 Room
- ClassSession N:1 Teacher
- ClassSession N:1 ClassTemplate
- ClassSession 1:N Attendance
- Invoice N:1 Student or FamilyGroup
- Invoice 1:N InvoiceLineItem

- Payment N:1 Invoice
- Scholarship N:1 Student or FamilyGroup

3.3 Sample Prisma Schema (Abbreviated)

```
// prisma/schema.prisma

model User {
  id           String    @id @default(cuid())
  email        String    @unique
  name         String
  role         UserRole  @default(PARENT)
  student      Student?
  guardian     Guardian?
  teacher      Teacher?
  createdAt    DateTime  @default(now())
}

model FamilyGroup {
  id        String     @id @default(cuid())
  name      String
  students  Student[]
  guardians Guardian[]
  invoices  Invoice[]
}

model Student {
  id            String       @id @default(cuid())
  firstName     String
  lastName      String
  dateOfBirth   DateTime?
  familyGroup   FamilyGroup? @relation(fields: [familyGroupId], references: [id])
  familyGroupId String?
  guardian      Guardian?    @relation(fields: [guardianId], references: [id])
  guardianId    String?
  enrollments   Enrollment[]
  scholarships  Scholarship[]
  invoices      Invoice[]
}

model ClassSession {
  id          String       @id @default(cuid())
  template    ClassTemplate @relation(fields: [templateId], references: [id])
  templateId  String
  teacher     Teacher      @relation(fields: [teacherId], references: [id])
  teacherId   String
  room        Room         @relation(fields: [roomId], references: [id])
  roomId      String
  startTime   DateTime
  endTime     DateTime
  maxCapacity Int
  enrollments Enrollment[]
  attendance  Attendance[]
}
```

4. Feature Requirements

4.1 Authentication & Role System

Requirements

- Email/password login with secure password hashing (bcrypt)
- OAuth login via Google (optional but recommended for parents)
- Role-based access control (RBAC): SUPER_ADMIN, ADMIN, TEACHER, PARENT, STUDENT
- Invitation-based teacher onboarding (admin sends invite link)
- Parent self-registration tied to student enrollment flow
- Session management with secure httpOnly cookies
- Password reset via email link
- Two-factor authentication (Phase 2)

4.2 Student & Family CRM

Requirements

- Create, read, update, archive student profiles
- Fields: name, DOB, photo, contact info, emergency contact, medical notes, skill level, dance styles
- Assign student to a FamilyGroup
- Map guardian <-> student relationships (multiple guardians per student supported)
- Full engagement history per student: enrollment history, attendance record, payment history, scholarship history, notes timeline
- Teacher-facing student view: attendance, enrolled classes, basic notes (no financial data)
- Admin-facing student view: full history including billing, outstanding balances, scholarship details
- Search and filter students by name, class, enrollment status, family group, balance
- Bulk actions: enroll group, send email to cohort, generate invoices for group
- Student tags and custom fields for studio-specific metadata

4.3 Teacher Management

Requirements

- Teacher profiles: bio, specialties, certifications, photo, contact info
- Assign teachers to class sessions
- Teacher dashboard: view own upcoming classes, student roster per class, attendance marking
- Teacher cannot view or modify billing, invoices, or admin financials
- Admin can view teacher utilization: hours taught, classes per week, student counts
- Teacher engagement history: all sessions taught, attendance records submitted

4.4 Room Management

Requirements

- CRUD operations for studio rooms/spaces
- Fields: name, description, capacity, floor type (optional), amenities notes
- Rooms displayed in scheduling UI as selectable resources
- Conflict detection: prevent double-booking a room at same time
- Admin view showing room utilization stats (hours used, % capacity filled)

4.5 Class Scheduling & Management

Requirements

- Class Templates: define reusable class types with name, description, dance style, skill level, default duration, default price
- Class Sessions: schedule instances from templates with specific date/time, teacher, room, max capacity
- Recurring sessions: set weekly/biweekly recurring schedule with end date
- Calendar view (week/month) for admin showing all sessions across rooms
- Teacher calendar view showing only their own sessions
- Enrollment cap enforcement: prevent over-enrollment, show waitlist option
- Waitlist management: auto-notify next on waitlist when spot opens
- Session cancellation with auto-notification to enrolled students/parents
- Session notes field for teachers (lesson notes, recital prep notes)

4.6 Enrollment Management

Requirements

- Admin can manually enroll or remove a student from any class session
- Parent can enroll their student(s) via parent portal
- Adult student can self-enroll via student portal
- Public registration flow: visitor browses schedule, initiates enrollment (requires account creation or login)
- Enrollment triggers invoice generation (or adds to next billing cycle)
- Unenrollment policy: configurable (e.g., must unenroll X days before class for refund)
- Enrollment confirmation email sent automatically

4.7 Attendance Tracking

Requirements

- Teacher opens attendance view for a specific class session
- Student list pre-populated from enrollments
- Mark each student: Present, Absent, Late, Excused Absent
- Teacher submits attendance — locked after submission (admin can unlock/override)
- Admin can view attendance reports by student, class, teacher, or date range
- Attendance percentage displayed on student profile
- Low attendance alert: flag students below configurable threshold (e.g., below 70%)

4.8 Tuition & Custom Billing

Requirements

- Per-class custom pricing: each ClassTemplate can have its own base price
- Pricing override per session if needed
- Family pricing rules: configurable discount logic for family groups
- Example rule types: '2nd student in family pays 50%', 'family cap of $X/month', 'flat rate for N or more family members'
- Pricing rules stored as PricingRule records linked to FamilyGroup or globally
- Custom one-off charges: add arbitrary line items to a student or family invoice
- Billing cycles: monthly or per-session (configurable per studio)
- Invoice generation: auto-generate monthly invoices or trigger manually
- Invoice statuses: Draft, Sent, Paid, Overdue, Void
- Invoice PDF generation and email delivery
- Admin can manually void or adjust invoices

4.9 Online Payment Portal (Parents / Students)

Requirements

- Parent/student logs in and sees outstanding invoices
- Pay invoice via Stripe (credit/debit card, ACH optional)
- Payment receipt emailed automatically
- Saved payment methods (Stripe Customer + PaymentMethod)
- Auto-pay enrollment: parent opts in to automatic monthly charge
- Payment history visible to parent/student
- Failed payment retry logic and notification
- Stripe Customer Portal link for self-service card management

4.10 Invoicing

Requirements

- Invoices generated per student or per family group
- Line items: tuition per class, discounts, scholarship credits, custom charges, late fees
- Discount line items show applied pricing rules (transparency)
- PDF invoice with studio branding, logo, contact info
- Email invoice delivery with pay-now link
- Partial payments tracked against invoice balance
- Aging report: invoices by days outstanding
- Admin bulk invoice generation for all active students for a billing period

4.11 Scholarship Tracking

Requirements

- Admin can create and assign scholarships to a student or family group
- Scholarship types: percentage discount, fixed dollar amount, full tuition waiver
- Scholarship fields: name, donor/sponsor name (optional), amount/%, applicable classes (all or specific), start date, end date, notes
- Scholarships automatically applied as line item credit when invoices are generated
- Scholarship history on student and family group profiles
- Admin report: total scholarship value awarded by period

4.12 Financial Planning & P&L Dashboard

Requirements

- Revenue projections: based on enrolled students x tuition rates for upcoming billing periods
- Actual vs projected revenue comparison
- Expense tracking: admin can log expense categories (rent, utilities, teacher pay, supplies, etc.)
- Net P&L summary by month and year
- Cash flow forecast: expected receivables minus logged expenses
- Scholarship impact report: total dollars in scholarship credits reducing revenue
- Outstanding receivables: total unpaid invoice balances
- Revenue by class, by teacher, by room, by time period (drill-down filters)
- Enrollment revenue modeling: 'what if' scenario for adding/removing classes or adjusting pricing
- Export financial reports to CSV/Excel

4.13 Admin Statistics & Utilization Dashboard

Requirements

- Total active students, new enrollments this month, unenrollments this month
- Classes by enrollment fill rate (% of capacity used)
- Room utilization: hours booked vs available hours per room
- Teacher hours: sessions and total hours per teacher per period
- Attendance rate by class, teacher, and studio overall
- Churn indicators: students who haven't attended in X weeks
- Revenue per student average, revenue per class average
- Top performing classes (revenue, attendance, waitlist size)
- Data range selectors (this week, this month, custom range)

4.14 Public-Facing Website (MVP)

Requirements

- Homepage: studio branding, hero section, brief about
- Schedule page: public-facing class schedule with filter by style/level/day
- Class detail page: description, teacher bio, price, available spots, enroll CTA

- Registration / signup flow: create account (parent or adult student), add students, select classes, pay
- About / Teachers page: teacher profiles and bios
- Contact page with form or contact info
- SEO basics: meta titles, Open Graph tags, sitemap
- Responsive design (mobile-first)

4.15 Embeddable Schedule & Signup Widget

Requirements

- A JavaScript snippet that studio owners can paste into any external website
- Renders a styled class schedule table or calendar view
- Clicking a class opens enrollment flow (hosted on the platform domain)
- Widget is responsive and customizable via data attributes (colors, filters)
- Implemented as a standalone Next.js page rendered in an iframe
- Example embed code: <script src='https://app.domain.com/widget.js' data-studio='studio-slug'></script>

5. User Stories by Role

5.1 Admin / Owner Stories

1. As an admin, I can create and manage rooms so I can organize scheduling by physical space.
2. As an admin, I can create class templates and schedule recurring sessions with room and teacher assignments.
3. As an admin, I can enroll or unenroll students from any class and see enrollment history.
4. As an admin, I can create family groups and link students and guardians to them.
5. As an admin, I can configure family pricing rules so discounts apply automatically on invoices.
6. As an admin, I can generate monthly invoices for all families and send them via email.
7. As an admin, I can issue scholarships and track their impact on revenue.
8. As an admin, I can view a P&L dashboard showing projected vs actual revenue and expenses.
9. As an admin, I can see class utilization, teacher hours, and attendance rates in the stats dashboard.
10. As an admin, I can export financial and enrollment reports to CSV.

5.2 Teacher Stories

11. As a teacher, I can view my upcoming class schedule with room assignments.
12. As a teacher, I can view the student roster for each of my classes.
13. As a teacher, I can mark attendance for each class session and submit it.
14. As a teacher, I can view individual student profiles including their attendance history in my classes.
15. As a teacher, I cannot access billing, invoices, or admin financial data.

5.3 Parent / Guardian Stories

16. As a parent, I can create an account and add my children as students.
17. As a parent, I can browse the class schedule and enroll my children in classes.
18. As a parent, I can view outstanding invoices and pay them online via credit card.
19. As a parent, I can enroll in auto-pay so tuition is automatically charged monthly.
20. As a parent, I can view my children's attendance history and class enrollment history.
21. As a parent, I can update my payment method and view past payment receipts.

5.4 Public / Visitor Stories

22. As a visitor, I can browse the public class schedule without creating an account.
23. As a visitor, I can click a class and see its details, price, and availability.
24. As a visitor, I can start an enrollment flow that prompts me to create an account.
25. As a visitor on an external website, I can see an embedded schedule widget and click to register.

6. Implementation Plan

The following phased plan is designed for AI-assisted development in Cursor. Each phase builds on the last and represents a coherent, shippable increment.

Phase 1: Foundation (Weeks 1–2)

- Initialize Next.js 14 project with TypeScript, Tailwind CSS, and shadcn/ui
- Set up Prisma with PostgreSQL (Supabase or Neon)
- Implement full schema from Section 3
- Configure NextAuth.js with email/password and role-based middleware
- Build role-aware layout shell: Sidebar nav, top bar, role-gated route protection
- Seed database with demo data (studio, rooms, teachers, students, classes)

Phase 2: CRM Core (Weeks 3–4)

- Build Student CRUD pages (admin + teacher views)
- Build Guardian management and family group linking
- Build Teacher profile management
- Implement CRM history timeline (enrollment, attendance, payment events)
- Build search and filter UI for students and families

Phase 3: Scheduling & Rooms (Weeks 5–6)

- Build Room CRUD admin interface
- Build Class Template CRUD
- Build Class Session scheduler with FullCalendar integration
- Implement recurring session generation
- Build room conflict detection logic
- Build teacher and admin calendar views

Phase 4: Enrollment (Week 7)

- Admin enrollment management: enroll/unenroll student from session
- Enrollment cap and waitlist logic
- Enrollment confirmation email (via Resend)
- Parent portal enrollment flow

Phase 5: Attendance (Week 8)

- Teacher attendance marking UI
- Attendance submission and locking
- Admin attendance reports and override capability
- Attendance stats on student profile

Phase 6: Billing & Invoicing (Weeks 9–10)

- Implement PricingRule system with family discount logic
- Invoice generation engine (per student/family, per billing period)
- Invoice CRUD and status management
- PDF invoice generation with @react-pdf/renderer
- Scholarship creation and auto-application to invoices
- Bulk invoice generation for billing period

Phase 7: Payments (Weeks 11–12)

- Stripe integration: Customer creation, PaymentIntent, Checkout
- Parent payment portal: view invoices, pay online
- Auto-pay with Stripe Subscriptions or scheduled intents
- Payment success/failure webhooks and invoice status updates
- Receipt email on successful payment
- Stripe Customer Portal for card management

Phase 8: Financial Dashboard & Stats (Week 13)

- Admin P&L dashboard with revenue projections vs actuals
- Expense tracking module
- Class utilization and room utilization reports
- Teacher hours and attendance rate dashboard
- CSV/Excel export of all reports

Phase 9: Public Website (Week 14)

- Public homepage with studio branding
- Public class schedule page with filters
- Class detail and enrollment CTA
- Teacher bios page
- Public registration and enrollment onboarding flow
- SEO and mobile responsiveness

Phase 10: Embeddable Widget & Polish (Week 15)

- Build iframe-embeddable schedule widget
- Widget customization via data attributes
- End-to-end QA across all roles
- Accessibility audit (WCAG 2.1 AA)
- Security review: RBAC audit, API route protection
- Production deployment to Vercel + Supabase

7. Recommended Project Structure

```
/
├── app/                          # Next.js App Router
│   ├── (public)/                 # Public-facing pages (no auth)
│   │   ├── page.tsx              # Homepage
│   │   ├── schedule/page.tsx     # Public schedule
│   │   ├── classes/[id]/page.tsx
│   │   ├── register/page.tsx     # Signup / enrollment onboarding
│   │   └── layout.tsx            # Public nav + footer
│   ├── (app)/                    # Authenticated app shell
│   │   ├── admin/                # Admin-only routes
│   │   │   ├── dashboard/
│   │   │   ├── students/
│   │   │   ├── families/
│   │   │   ├── teachers/
│   │   │   ├── classes/
│   │   │   ├── rooms/
│   │   │   ├── schedule/
│   │   │   ├── billing/
│   │   │   ├── scholarships/
│   │   │   └── reports/
│   │   ├── teacher/              # Teacher-only routes
│   │   │   ├── dashboard/
│   │   │   ├── classes/
│   │   │   └── attendance/
│   │   ├── parent/               # Parent portal
│   │   │   ├── dashboard/
│   │   │   ├── students/
│   │   │   ├── schedule/
│   │   │   └── billing/
│   │   └── layout.tsx            # App shell with sidebar
│   ├── api/                      # API Route Handlers
│   │   ├── auth/
│   │   ├── students/
│   │   ├── classes/
│   │   ├── enrollment/
│   │   ├── attendance/
│   │   ├── invoices/
│   │   ├── payments/
│   │   └── webhooks/stripe/
│   └── widget/                   # Embeddable widget page
├── components/
│   ├── ui/                       # shadcn/ui primitives
│   ├── forms/                    # Reusable form components
│   ├── tables/                   # Data table components
│   ├── charts/                   # Recharts wrappers
│   ├── calendar/                 # FullCalendar wrappers
│   └── pdf/                      # React PDF templates
├── lib/
│   ├── prisma.ts                 # Prisma client singleton
│   ├── auth.ts                   # NextAuth config
│   ├── stripe.ts                 # Stripe client
│   ├── email.ts                  # Resend/email helpers
│   ├── billing.ts                # Invoice generation logic
│   └── permissions.ts            # RBAC helpers
├── prisma/
│   ├── schema.prisma
│   └── seed.ts
├── types/                        # Global TypeScript types
├── hooks/                        # Custom React hooks
├── .env.local
├── next.config.ts
└── tailwind.config.ts
```

8. Environment Variables

The following environment variables must be configured in .env.local and in the Vercel project settings:

```
# Database
DATABASE_URL="postgresql://..."

# Auth (NextAuth)
NEXTAUTH_URL="https://yourdomain.com"
NEXTAUTH_SECRET="<random-secret>"

# Stripe
STRIPE_PUBLISHABLE_KEY="pk_live_..."
STRIPE_SECRET_KEY="sk_live_..."
STRIPE_WEBHOOK_SECRET="whsec_..."

# Email (Resend)
RESEND_API_KEY="re_..."
EMAIL_FROM="noreply@yourstudio.com"

# Storage (Supabase)
NEXT_PUBLIC_SUPABASE_URL="https://..."
NEXT_PUBLIC_SUPABASE_ANON_KEY="..."
SUPABASE_SERVICE_ROLE_KEY="..."

# App Config
NEXT_PUBLIC_APP_URL="https://yourdomain.com"
NEXT_PUBLIC_STUDIO_SLUG="my-dance-studio"
```

9. Key Business Logic Specifications

9.1 Family Pricing Rule Engine

When generating an invoice for a FamilyGroup, the billing engine must:

26. Collect all active enrollments for all students in the family group for the billing period.
27. Sort enrollments by price descending (highest price first).
28. Apply base price to the first (most expensive) enrollment per family.
29. Apply the family discount rule to subsequent enrollments. Rules can be: percent reduction, fixed price override, or flat family cap.
30. Apply any scholarship credits as negative line items after discounts.
31. Round all monetary values to 2 decimal places using banker's rounding.

9.2 Enrollment & Capacity

Enrollment must enforce capacity at the ClassSession level:

- enrolledCount = COUNT(Enrollment WHERE sessionId = X AND status = ACTIVE)
- If enrolledCount >= maxCapacity, enrollment is rejected with WaitlistOffer response
- Waitlist entries stored with position number; re-ordered on unenrollment
- Admin can override capacity limit with an explicit flag

9.3 Attendance Locking

- Attendance is OPEN when session endTime has passed and teacher has not submitted
- Attendance is LOCKED after teacher submits
- Only ADMIN or SUPER_ADMIN role can unlock attendance for correction
- All unlocks logged to AuditLog with actor ID and reason

9.4 Invoice Generation Trigger

- Manual trigger: admin clicks 'Generate Invoices for [Month]'
- Automatic trigger: background job runs on 1st of each month (configurable)
- System checks for existing draft invoice for same student/period before creating duplicate
- Generated invoices start in DRAFT status; admin reviews and sends (status -> SENT)
- Sending an invoice emails PDF to guardian/student and sets sent timestamp

9.5 Stripe Webhook Handling

- payment_intent.succeeded -> mark Payment as PAID, mark Invoice as PAID, send receipt email
- payment_intent.payment_failed -> mark Payment as FAILED, send failure notification, trigger retry
- customer.subscription.deleted -> disable auto-pay, notify admin
- All webhook events logged to AuditLog

10. Non-Functional Requirements

Requirement	Target	Notes
Performance	< 2s page load (P95)	Use Next.js SSR caching, DB query optimization
Uptime	99.9% availability	Vercel SLA + Supabase managed DB
Security	OWASP Top 10 mitigated	Input validation, RBAC on all API routes, no PII in logs
Data Residency	US region default	Supabase US East, Vercel US East
PCI Compliance	Stripe handles card data	No raw card data ever touches app servers
FERPA Awareness	Student data handled carefully	Access controls, minimal data exposure to teachers
Accessibility	WCAG 2.1 AA	shadcn/ui is ARIA-compliant; test with axe-core
Mobile	Fully responsive	Mobile-first Tailwind design for parent portal and public pages
Scalability	Support 10k students per studio	Indexed DB queries, pagination on all lists
Audit Trail	Immutable AuditLog	All creates, updates, deletes, and role actions logged

11. Notes for AI-Assisted Development (Cursor)

The following guidance is specifically to assist AI code generation tools like Cursor in producing high-quality, consistent code for this project.

11.1 Code Conventions

- All components use TypeScript with strict mode enabled
- All API routes must validate input using Zod schemas before touching the database
- All database queries use Prisma — no raw SQL except for complex aggregation queries
- All Prisma calls happen in server components, server actions, or API route handlers — never in client components
- Use the repository pattern: db access functions in /lib/db/*.ts, called from API routes
- All monetary values stored as integers in cents (e.g., $25.00 stored as 2500)
- All dates stored as UTC; display in studio local timezone via client-side formatting

11.2 Prompt Guidance for Cursor

- When building forms, always generate: Zod schema, React Hook Form integration, API route handler, and Prisma mutation together
- When building list/table pages, always include: pagination, search, and filter UI
- When building role-gated routes, always add getServerSession check at the top of the server component or API handler
- When building financial calculations, write pure functions in /lib/billing.ts with unit tests
- When integrating Stripe, always handle both success and webhook confirmation — never trust only the client redirect

11.3 Initial Cursor Prompt Suggestion

To begin development, use the following prompt in Cursor after cloning the repo:

"Initialize a Next.js 14 project with TypeScript, Tailwind CSS, shadcn/ui, Prisma (PostgreSQL), and NextAuth.js. Create the full Prisma schema for a Dance Studio Management Platform as specified in the PRD. Set up role-based middleware for ADMIN, TEACHER, and PARENT roles. Generate the database seed file with sample data."

12. Appendix: Glossary

Term	Definition
Class Template	A reusable class type definition (e.g., 'Beginner Ballet') used to create multiple scheduled sessions
Class Session	A specific scheduled instance of a class template at a particular time, room, and with a specific teacher
Enrollment	The active relationship between a Student and a Class Session
FamilyGroup	A billing and relational unit grouping one or more guardians and their associated students
PricingRule	A configurable discount rule applied to family group invoices (e.g., 50% off 2nd student)
Scholarship	A discount or credit awarded to a student or family, reducing their invoice total
Invoice	A billing document listing charges, discounts, and credits for a student or family for a period
Attendance Record	A per-student, per-session record of presence status submitted by a teacher
Room	A named physical space within the studio used for scheduling class sessions
Widget	An embeddable JavaScript snippet showing the public schedule on external websites
AuditLog	An immutable record of all significant platform actions for compliance and debugging
RBAC	Role-Based Access Control — permissions determined by the user's assigned role

— End of Document —

Dance Studio Management Platform PRD v1.0 | Confidential